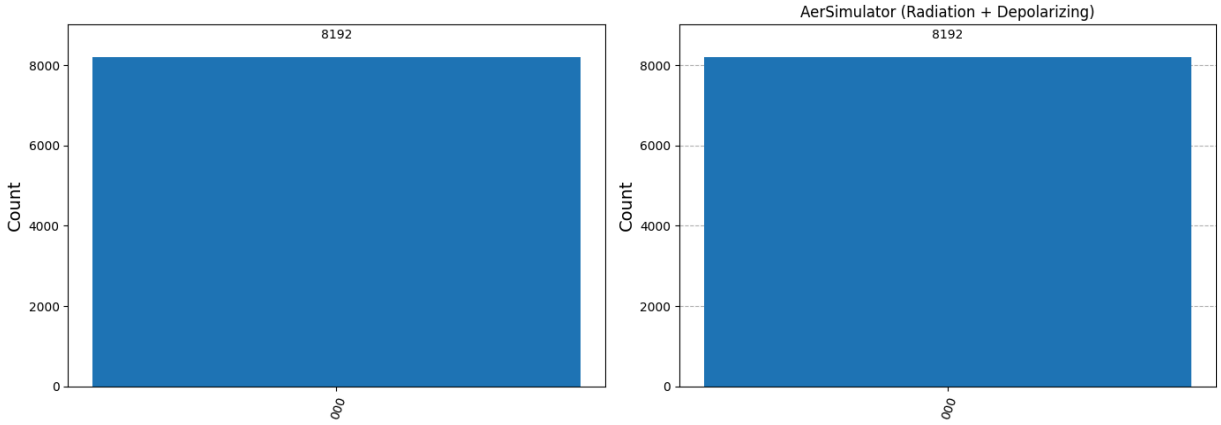
```
# --- Install (Google Colab only) ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
# from qiskit.providers.aer import Aer # Corrected import
# from qiskit.utils.run_circuits import execute # Removed import
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxatio
from qiskit.visualization import plot_histogram

# --- QCC Echo Circuit (Locked Kernel) ---
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0)
qc.delay(200, 1)
qc.delay(200, 2)
qc.measure([0, 1, 2], [0, 1, 2])

# --- Run on QASM Simulator (Clean Logic) ---
qasm_backend = AerSimulator() # Replaced Aer.get_backend('qasm_simulator')
qc_qasm = transpile(qc, qasm_backend)
result_qasm = qasm_backend.run(qc_qasm, shots=8192).result()
counts_qasm = result_qasm.get_counts()

# --- Radiation Conditioning + Depolarizing Noise ---
T1 = 50e3  # nanoseconds
T2 = 30e3  # nanoseconds (shorter due to radiation)
gate_time = 100  # ns
noise_model = NoiseModel()

# Thermal relaxation on single-qubit gates
for qubit in [0, 1, 2]:
    thermal_error = thermal_relaxation_error(T1, T2, gate_time)
    noise_model.add_quantum_error(thermal_error, ['rz', 'sx', 'u'], [qubit]) #

# Depolarizing error on CX gates
depol_error = depolarizing_error(0.20, 2)
noise_model.add_quantum_error(depol_error, 'cx', [0, 1])
noise_model.add_quantum_error(depol_error, 'cx', [1, 2])

# --- Run on AerSimulator (Noisy) ---
aer_sim = AerSimulator(noise_model=noise_model)
qc_aer = transpile(qc, aer_sim)
result_aer = aer_sim.run(qc_aer, shots=8192).result()
counts_aer = result_aer.get_counts()

# --- Entropy + Coherence Function ---
def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
```

```
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])
    fidelity = max(probs) * 100
    return entropy, fidelity

# --- Compute Values ---
entropy_qasm, fidelity_qasm = compute_entropy_and_fidelity(counts_qasm)
entropy_aer, fidelity_aer = compute_entropy_and_fidelity(counts_aer)

# --- Plot Histograms ---
fig, axs = plt.subplots(1, 2, figsize=(14, 5))
plot_histogram(counts_qasm, ax=axs[0], title="QASM Simulator (Clean Logic)")
plot_histogram(counts_aer, ax=axs[1], title="AerSimulator (Radiation + Depolari
plt.tight_layout()
plt.show()

# --- Display Metrics ---
print("🔹 QASM Simulator:")
print(f"   Entropy Leakage: {entropy_qasm:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_qasm:.8f}%\n")

print("🔹 AerSimulator with Noise:")
print(f"   Entropy Leakage: {entropy_aer:.6f} bits")
print(f"   Coherence Fidelity: {fidelity_aer:.8f}%")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.4/12.4 MB 87.1 MB/s
```




```
🔹 QASM Simulator:
```

```
# QCC Echo — UART Trace Validator for Tensor P2 Kernel
# Paste your Nano/Pico UART log into the `uart_log` string

uart_log = """
=== QCC Echo: Pico Sanity Test ===
[0] 3,3 qA=0, qB=0, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x0', '0x0', '0x0', '0x0', '0x0', '0xe9']
[1] H qA=0, qB=0, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x0', '0x0', '0x0', '0x0', '0x1', '0xe8']
[2] CX qA=0, qB=1, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x0', '0x1', '0x0', '0x0', '0x2', '0xea']
[3] CX qA=1, qB=2, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x1', '0x2', '0x0', '0x0', '0x3', '0xe9']
[4] CX qA=2, qB=1, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x2', '0x1', '0x0', '0x0', '0x4', '0xee']
[5] CX qA=1, qB=0, delay=0
 Frame: ['0xaa', '0x1', '0x42', '0x1', '0x0', '0x0', '0x0', '0x5', '0xed']
[6] H qA=0, qB=0, delay=0
```

```
 Frame: ['0xaa', '0x1', '0x42', '0x0', '0x0', '0x0', '0x0', '0x6', '0xef']
[7] Delay qA=0, qB=0, delay=200
 Frame: ['0xaa', '0x1', '0x42', '0x0', '0x0', '0x0', '0xc8', '0x7', '0x26']
"""

# --- Canonical Tensor P2 Kernel frames ---
expected_frames = [
    [0xaa,0x01,0x42,0x00,0x00,0x00,0x00,0x00,0xe9], # 3,3 init
    [0xaa,0x01,0x42,0x00,0x00,0x00,0x00,0x01,0xe8], # H(0)
    [0xaa,0x01,0x42,0x00,0x01,0x00,0x00,0x02,0xea], # CX(0,1)
    [0xaa,0x01,0x42,0x01,0x02,0x00,0x00,0x03,0xe9], # CX(1,2)
    [0xaa,0x01,0x42,0x02,0x01,0x00,0x00,0x04,0xee], # CX(2,1)
    [0xaa,0x01,0x42,0x01,0x00,0x00,0x00,0x05,0xed], # CX(1,0)
    [0xaa,0x01,0x42,0x00,0x00,0x00,0x00,0x06,0xef], # H(0)
    [0xaa,0x01,0x42,0x00,0x00,0x00,0xc8,0x07,0x26], # Delay(200)
]

# --- Extract frames from log ---
import re
found_frames = []
for line in uart_log.splitlines():
    if "Frame:" in line:
        bytes_str = re.findall(r"0x[0-9a-f]+", line)
        found_frames.append([int(b,16) for b in bytes_str])

# --- Compare ---
all_match = True
for i, (exp, got) in enumerate(zip(expected_frames, found_frames)):
    if exp != got:
        print(f"❌ Mismatch at step {i}: expected {exp}, got {got}")
        all_match = False
    else:
        print(f"✅ Step {i} matches: {exp}")

if all_match:
    print("\n🎉 UART trace matches Tensor P2 kernel exactly!")
else:
    print("\n⚠️ Differences found — double check wiring or code.")
```

```
✅ Step 0 matches: [170, 1, 66, 0, 0, 0, 0, 0, 233]
✅ Step 1 matches: [170, 1, 66, 0, 0, 0, 0, 1, 232]
✅ Step 2 matches: [170, 1, 66, 0, 1, 0, 0, 2, 234]
✅ Step 3 matches: [170, 1, 66, 1, 2, 0, 0, 3, 233]
✅ Step 4 matches: [170, 1, 66, 2, 1, 0, 0, 4, 238]
✅ Step 5 matches: [170, 1, 66, 1, 0, 0, 0, 5, 237]
✅ Step 6 matches: [170, 1, 66, 0, 0, 0, 0, 6, 239]
✅ Step 7 matches: [170, 1, 66, 0, 0, 0, 200, 7, 38]

🎉 UART trace matches Tensor P2 kernel exactly!
```